

Algorithm Design and Analysis

HUS - HKII,
2025-2026

Assignment 3

Lecturer: Nguyễn Thị Hồng Minh - Đặng Thị Oanh
Trần Bá Tuấn - Trần Anh Minh

§ RECURSION §

Section 1: Objectives

- Students need to grasp the basic knowledge of recursion and recursive algorithm schemes.
- Students understand how to analyze recursive algorithms in terms of correctness and complexity.
- Students clearly understand how to solve recurrence relations using the substitution method and the Master Theorem to determine algorithm complexity.
- Students learn about recursion elimination and some forms of recursion elimination presented in the theoretical lecture.

Section 2: Practice

(1) Quy cách nộp bài

- Nộp bài bằng file văn bản hoặc ảnh chụp scan thành PDF nếu bài làm viết tay ra giấy (khuyến khích sinh viên thực hiện trình bày bài làm bằng Latex).
- Chương trình viết bằng ngôn ngữ Java hoặc Python. Mã nguồn chương trình, file dữ liệu (nếu có) được đặt trong cùng thư mục, kèm theo hướng dẫn cách thực hiện chương trình nếu cần.
- Tất cả các file liên quan tới bài tập để trong thư mục tên Hw3_MaSinhVien_Hovaten, thư mục được nén thành file.zip cùng tên thư mục.
- Khi hết hạn nộp bài, các bài làm sẽ công bố để thực hiện chấm chéo bài tập.
- Sinh viên không nộp bài sẽ nhận điểm 0 bài tập tuần.
- Sinh viên CÓ GIAN LẬN trong nộp bài tập sẽ bị DÌNH CHỈ môn học (điểm 0 cho tất cả các điểm thành phần).

(2) Exercises

- Briefly introduce recursion, recursive algorithm schemes, and methods for solving recurrence relations to students.
- **(Master Theorem)** If n is a power of b , the solution to the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ aT\left(\frac{n}{b}\right) + n^d & \text{if } n > 1, a \geq 1, b > 1, d \geq 0 \end{cases}$$

is

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Example 1. Determine the complexity of the following recurrence relation:

$$T(n) = \begin{cases} 2T(n-1) - 1, & \text{if } n > 0. \\ 1, & \text{otherwise.} \end{cases}$$

Example 2. Determine the complexity of the following recurrence relation:

$$T(n) = \begin{cases} 3T(n-1), & \text{if } n > 0. \\ 1, & \text{otherwise.} \end{cases}$$

Example 3. Solve the recurrence relation to determine the algorithm's time complexity.

1. $T(1) = 1$, and for all $n \geq 2$ a power of 2, $T(n) = 2T\left(\frac{n}{2}\right) + 6n - 1$.
2. $T(1) = 2$, and for all $n \geq 2$ a power of 3, $T(n) = 4T\left(\frac{n}{3}\right) + 3n - 5$.
3. $T(1) = 1$, and for all $n \geq 2$ a power of 2, $T(n) = 3T\left(\frac{n}{2}\right) + n^2 - n$.

Exercise 1. Solve the recurrence relation to determine the algorithm's time complexity.

1. **Exercise 2.4, page 102-105, Anany's book.**
2. **Select and solve at least four of the following exercises.**
 - $T(1) = 1$, and for all $n \geq 2$, $T(n) = 3T(n-1) + 2$.
 - $T(1) = 3$, and for all $n \geq 2$, $T(n) = T(n-1) + 2n - 3$.
 - $T(1) = 1$, and for all $n \geq 2$, $T(n) = 2T(n-1) + n - 1$.
 - $T(1) = 5$, and for all $n \geq 2$, $T(n) = 2T(n-1) + 3n + 1$.

- $T(1) = 4$, and for all $n \geq 2$ a power of 2, $T(n) = 2T\left(\frac{n}{2}\right) + 3n + 2$.
- $T(1) = 1$, and for all $n \geq 2$ a power of 6, $T(n) = 6T\left(\frac{n}{6}\right) + 2n + 3$.
- $T(1) = 3$, and for all $n \geq 2$ a power of 6, $T(n) = 6T\left(\frac{n}{6}\right) + 3n - 1$.
- $T(1) = 3$, and for all $n \geq 2$ a power of 3, $T(n) = 4T\left(\frac{n}{3}\right) + 2n - 1$.
- $T(1) = 4$, and for all $n \geq 2$ a power of 2, $T(n) = 3T\left(\frac{n}{2}\right) + n^2 - 2n + 1$.
- $T(1) = 1$, and for all $n \geq 2$ a power of 2, $T(n) = 3T\left(\frac{n}{2}\right) + n - 2$.

Exercise 2. *Recursive and Recursion Elimination.*

Write recursive and non-recursive (recursion elimination) programs. Provide an evaluation and comparison between recursion and recursion elimination (the use of charts is encouraged) regarding the growth of program execution time with respect to the input size for the problems:

1. Print the binary representation of an integer.
2. Factor an integer into its prime factors.
3. Find the n -th Fibonacci number.
4. The Tower of Hanoi problem.

Exercise 3. *Problem Formulation, Algorithm Design, Analysis, and Implementation*

Formulate at least two problems. Analyze each problem, design an algorithm, prove its correctness, evaluate its time complexity, and implement a program to illustrate the recursion technique.

Some suggested problems:

Knapsack Problem: There are n items, each with weight P_i and value V_i ($i = 1, \dots, n$). A knapsack can carry a maximum weight of M . Determine the maximum total value of the items that can be placed in the knapsack. Also indicate one possible selection of items.

Reward Distribution Problem: There are m rewards to be distributed to n outstanding students ranked from 1 to n . How many ways are there to distribute the rewards such that the following conditions are satisfied:

- (i) A higher-ranked student receives no fewer rewards than a lower-ranked student;
- (ii) All m rewards must be distributed among the students.

Polygon Triangulation Problem: Given a convex polygon with n vertices, divide the polygon into triangles using non-intersecting diagonals. Find the triangulation with the minimum total length of the diagonals.